Inventory Import Review - detailed recording guide (DRAFT)

Not Studio approved. This is a provisional plan, not a record of completed work. Use it only after the idea is approved, the actual permitted inputs are supplied, and this guide is reconciled with those files. Proposed control/function names are explicit; no source line numbers are invented. Add verified line anchors during finalization. Keep this guide and all project materials off the recorded screen.

The SAY text is a cue for natural synchronized speech, not a requirement to recite mechanically. Every meaningful click, selection, scroll, edit, menu choice and app switch must name its app, exact target, purpose and observed result. If you need an extra action, narrate that actual action too. Never say a test passed because this guide predicts it. The expected checks below must be observed.

SAY	Green: speak just before and during the paired action. Use the cursor to point to the target as you name it.
ACTION	Blue: perform the stated visible operation. Pause about a second after clicks and let dialogs finish.
EDIT	Purple: enter only the specified code, one short segment at a time. Keep each continuous typing/mouse-idle interval under 45 seconds.
CHECK	Gold: compare the actual visible result with the stated expectation. This is not proof of a pass until you see it.
IF WRONG	Red: name the actual app, field or line, expected result, observed problem, correction and retest. Do not invent an error. Occasional corrected slips are allowed; restart if searching or repeated mistakes dominate.

0. Readiness and setup - OFF SCREEN

CHECK	Idea approval is required first. Use the same Studio task after approval. If the reviewer changes the Goal, app list, scope or materials, synchronize the proposal, opening document, TaskRecorder and guide before recording.
CHECK	Actual input 1.html and input 2.md were prepared through the permitted process and checked. The starter has parseSource, renderRows and reviewImport scaffolding; validateBatch returns an empty array; the review handler omits clearing rows. The source text, IDs and messages match this guide. No completed output exists in the recording folder.
ACTION	Use a fresh neutral folder such as /Users/Shared/StockReview_Record containing only the two actual inputs. Rehearse in a separate disposable folder; do not move a completed rehearsal output into the fresh recording folder. Keep both original inputs unchanged.
ACTION	Prepare a Google Docs document in Firefox containing only the exact approved Goal, with no heading, approval ID, project instructions, or additional proposal text. Use the allowed account setup and hide personal account names/photos. Keep only this opening document behind TaskRecorder. Do not use Notes or TextEdit for the opening.
ACTION	In Visual Studio Code, prepare the neutral folder with Explorer visible, readable text and word wrap for the brief. Prepare Firefox. Disable substantive AI assistance and visible assistant panels. Do not display Codex, Gemini, GitHub, Studio, these scripts or the odin repository path during the take.

CHECK	TaskRecorder 2.4.4: all required permissions green, microphone on and tested, correct screen selected, Task Description replaced with only the exact approved Goal, Used Apps matching approved Visual Studio Code and Firefox. Google Docs is opening-only, not extra substantive task work.
CHECK	Accessibility Inspector remains running with its window closed; the Dock icon indicator stays visible. Side Dock does not auto-hide, default cursor, visible menu-bar seconds, required display resolution, power and sufficient storage, notifications/PII hidden. Apply one permitted appearance change before recording. Keep the recording in a native macOS environment.
CHECK	Use ordinary windows, with the menu bar and Dock visible. Do not use full-screen or full-screen Split View. Rehearse normal window resizing and Firefox text selection before the actual take.
CHECK	Plan roughly 30-42 minutes including intro and epilogue; measure actual rehearsal rather than padding. Writer total range is 15:00-50:59. Coding typing stays within 30% of total time. No continuous typing/mouse inactivity over 45 seconds except in the epilogue. If the natural task cannot meet these limits, revise it before recording.

1. Brief Goal opening - under 20 seconds

ACTION	In TaskRecorder, click Start Recording. When recording starts, click its minimize control so only the Goal document in Firefox remains visible.
SAY	"I will be beginning my task soon. Here is the workflow I have in mind. I will inspect the inventory-import rules, implement batch validation in Visual Studio Code, and check the local preview in Firefox."
ACTION	Point briefly to the Goal paragraph. Close the opening document tab using its tab close button, then click Visual Studio Code in the side Dock. Announce this move as you do it. Do not read the full Goal aloud.
CHECK	The complete intro stays under 20 seconds. The visible Goal is exact and no approval ID, project page or private information appears.

2. Read the requirements - about 0:20-4:30

SAY	"In Visual Studio Code's Explorer on the left, I am clicking input 2.md. I want the import rules and expected outcomes before changing the page."
ACTION	Open input 2.md, pause and point to its title, then the format and business-rule headings.
SAY	"In the format section of input 2.md, each nonblank record has item ID, location and quantity separated by a vertical bar. This is a small plain-text format, not a general CSV parser with quoted fields."
ACTION	Point to the header item_id|location|quantity, the three-field rule, whitespace handling and physical line-number requirement. If below view, say you are scrolling down within the input 2.md editor to those rules before scrolling.

SAY	"In the business rules, item IDs and locations are required. IDs must be compared after trimming and uppercasing. Quantities use digits only, from zero through 9999. A blank string must not become a valid zero just because Number converts it to zero."
ACTION	Point individually to required values, duplicate normalization and quantity grammar/range. Keep the brief unchanged.
SAY	"At the batch rule in this same editor, any error rejects the whole preview. A later review must clear old rows and errors, so a rejected batch cannot appear alongside an earlier successful preview."
ACTION	Point to all-or-nothing rejection and valid-invalid-valid recovery. Scroll explicitly to this heading if needed.
SAY	"I am scrolling down in input 2.md to I01 through I12. These groups cover normal input, zero and leading zeros, duplicates, missing fields, invalid quantities, structural errors, safe text, recovery and bounds. I will compare the displayed rows, totals and error lines with these expectations."
ACTION	Scroll to the acceptance groups, point to each group heading, and locate I09's literal Store value and I10's three-review sequence. Do not claim a result yet.

3. Inspect the starter and save the output copy - about 4:30-8:00

SAY	"In Visual Studio Code's Explorer, I am clicking input 1.html to inspect the starter. I will preserve this source and create a separate output before editing."
ACTION	Open input 1.html and point to its tab. Scroll in this HTML editor to the body controls, naming the destination before the scroll.
SAY	"At the body of input 1.html, source is the labeled text area, validateBtn is Review import, status is the live result, errors is the error list, and rows is the preview table body. These are the elements the existing handlers use."
ACTION	Point to the five IDs and the local-review-only note. Do not edit the original.
SAY	"I am scrolling down in input 1.html to parseSource. It already checks the header and three fields, trims values, tracks physical line numbers and skips blank lines. I will retain those parsing decisions."
ACTION	Point to header comparison, field-count error and the record object with number, id, location, rawQuantity and quantity. Check these actual property names against the guide.
SAY	"I am moving farther down this HTML editor to validateBatch. It currently returns an empty array, so it does not reject invalid business values. Below it, renderRows already uses textContent for table cells, which I will preserve."
ACTION	Point to the empty validator, then document.createElement and td.textContent in the renderer. If you scroll between them, announce the function you are seeking.

SAY	"I am scrolling to reviewImport near the end of the script. It clears errors and combines parser and validator errors before rendering, but it never clears previous preview rows. That explains how an old preview could remain after a rejected attempt."
ACTION	Point to the existing errorsEl.replaceChildren, the combined errors array, the rejection return and renderRows call. Do not claim you observed the bug in Firefox yet.
SAY	"From Visual Studio Code's File menu, I am choosing Save As and naming the working copy final output 1.html in this neutral folder. This keeps input 1.html and input 2.md unchanged."
ACTION	Click File, then Save As. In the dialog, select the neutral folder and filename field, enter final output 1.html, and click Save. Announce each selection; hide any private path before the take, rather than repairing it on camera.
CHECK	Active tab is final output 1.html, both inputs are separate and unchanged, and the working copy still contains the incomplete starter. If the name or location is wrong, correct Save As before editing.

4. Observe both baseline defects in Firefox - about 8:00-11:00

SAY	"I am clicking Firefox in the side Dock, then File and Open File, to open final output 1.html. This copy is still unmodified, so I can observe the starter's behavior."
ACTION	Switch apps. Choose File > Open File, select the neutral task folder and final output 1.html, then click Open. Point to the local-only note and Review import button.
SAY	"In Firefox's Inventory rows text area, I am entering a header and two rows whose IDs differ only in capitalization. Under the brief, they should count as duplicate A101 records."
EDIT	Enter these three lines in the source text area: item_id|location|quantity / A101|North store|3 / a101|Loading bay|2. Each slash here means a line break; do not type slashes.
SAY	"I am clicking Review import below the text area. I expect this incomplete validator to accept the duplicate batch; I will describe the result that actually appears."
ACTION	Click the button, pause and point to the status and two preview rows.
CHECK	Intended baseline: two rows displayed, no duplicate error, and a Validated status. The row IDs remain A101 and a101. If this does not happen, stop and reconcile the actual starter with the guide before continuing; do not invent the expected baseline.
SAY	"In the same Firefox text area, I am replacing only the header's item_id with id, leaving the two data rows. I am clicking Review import again to observe the stale-preview defect."
ACTION	Click the first header field, select only item_id and type id. Click Review import, pause, and point first to the header error, then to the old table rows below it.
CHECK	Intended baseline: the header is rejected, but the earlier two preview rows remain. An old preview is not evidence of a successful current review. State the actual observed result.

SAY	"I am returning to Visual Studio Code through the side Dock. The observed issues are missing business validation and preview rows that survive a later rejection. I will fix them in the separate output file."
ACTION	Click Visual Studio Code, select final output 1.html and confirm the tab name before any edit.

5. Implement the batch validator - about 11:00-17:00

SAY	"In final output 1.html in Visual Studio Code, I am scrolling to function validateBatch(records). I am selecting only the return-empty-array statement inside that function, leaving its outer braces intact."
ACTION	Scroll in the output editor to the named function. Select only return []; and delete it. Do not remove the function declaration or closing brace.
SAY	"Inside validateBatch, I am creating one error list and one Set of normalized IDs. The Set will detect duplicate IDs in this review only."
EDIT	Type: `const errors = [], seen = new Set();` Then pause and point to both names.
SAY	"On the next line inside validateBatch, I am looping over every parsed record. I will collect all business issues rather than stopping after the first bad value."
EDIT	Type: `records.forEach(record => {` Place the cursor inside that callback, preserving or managing the editor's auto-inserted closing characters so the final code has one matching `});`.
CHECK	The outer function still has its own closing brace. Confirm actual auto-closing behavior from rehearsal; never type duplicate closing braces blindly.
SAY	"At the top of the record callback, I am uppercasing the ID before comparing it. The parser already trimmed the field. I am then creating a prefix with the record's physical source line number."
EDIT	Type `record.id = record.id.toUpperCase();` then `const prefix = 'Line ' + record.number + ': ';`. Pause and point to the uppercasing and number property.
SAY	"On the next line of this callback, I am rejecting an empty item ID. I will still inspect the other fields on that row."
EDIT	Type: `if (!record.id) errors.push(prefix + 'item ID is required.');`
SAY	"Below the required-ID check, I am checking the normalized ID against seen. I only test a nonempty ID, so an empty value is reported as missing rather than treated as a duplicate key."
EDIT	Type `if (record.id && seen.has(record.id)) {` then, inside that block, `errors.push(prefix + 'duplicate item ID ' + record.id + '.');` and close only that if block with `}`. Pause after each short entry.
SAY	"Immediately after the duplicate check, I am adding a nonempty ID to seen. I am also rejecting an empty location. This keeps ID identity separate from the quantity and location checks."

EDIT	Type `if (record.id) seen.add(record.id);` then `if (!record.location) errors.push(prefix + 'location is required.');`. Pause and point to each condition.
SAY	"Below the location check, I am validating the raw quantity string with a digits-only expression and the parsed number against 9999. This rejects blanks, signs, decimals and exponent syntax while allowing zero and leading zeros."
EDIT	Type `if (!/^\d+$/.test(record.rawQuantity) || record.quantity > 9999) {`. Inside it, type `errors.push(prefix + 'quantity must be an integer from 0 to 9999.');`. Close this if block with `}`. Pause and inspect the backslash-d, plus sign, rawQuantity property, and strict greater-than boundary.
SAY	"I am closing the per-record callback, then returning the collected errors from validateBatch. The existing review handler can now reject the entire batch before any preview is rendered."
EDIT	Close the callback with `});`, then type `return errors;` below it and inside the function. Leave the original function-closing brace intact.
CHECK	The complete validator matches the block below. This block is an offscreen reference; do not paste a completed solution from this guide during recording. Enter the short segments through the recorded work and understand each one.

```
function validateBatch(records) {
  const errors = [], seen = new Set();
  records.forEach(record => {
    record.id = record.id.toUpperCase();
    const prefix = 'Line ' + record.number + ': ';
    if (!record.id) errors.push(prefix + 'item ID is required.');
    if (record.id && seen.has(record.id)) {
      errors.push(prefix + 'duplicate item ID ' + record.id + '.');
    }
    if (record.id) seen.add(record.id);
    if (!record.location) errors.push(prefix + 'location is required.');
    if (!/^\d+$/.test(record.rawQuantity) || record.quantity > 9999) {
      errors.push(prefix + 'quantity must be an integer from 0 to 9999.');
    }
  });
  return errors;
}
```

IF WRONG	If there is a syntax underline, inspect the actual function in the output editor: callback closing `});`, nested if braces, quotes and property spelling. Name any real correction, make it on camera and verify it. Do not move on with an unexplained syntax error.

6. Clear stale rows, review and save - about 17:00-19:00

SAY	"I am scrolling down in Visual Studio Code's final output 1.html to function reviewImport. At its start, I am adding rowsEl.replaceChildren before parsing and before any early return. Every review will begin with an empty preview."
ACTION	Scroll to the named function and point to its opening brace and existing errorsEl.replaceChildren call. Click a new line immediately inside the function, before that existing call.
EDIT	Type exactly: `rowsEl.replaceChildren();`

CHECK	The handler now clears rows and errors on every click, then parses, validates, rejects on any error, and renders only after the error check. Do not put the new line inside the success branch; that would leave stale rows after failure.
SAY	"I am scrolling back to renderRows in this output file. I am keeping its document.createElement and td.textContent assignment, so a location that looks like HTML stays literal text. I am not changing the local-review-only result wording."
ACTION	Point to the safe cell assignment and the success/rejection messages. Scroll deliberately between these named regions if needed. Check no network or storage code was added.
SAY	"I am using Visual Studio Code's File menu and Save to save final output 1.html. I will test this exact saved file in Firefox."
ACTION	Click File > Save, pause and check the tab's unsaved indicator is gone. If using Command-S, announce the shortcut before pressing it.

7. Set up readable source-and-browser verification - about 19:00-20:30

SAY	"I am opening input 2.md in Visual Studio Code and arranging it on the left, with Firefox's output page on the right. This lets me copy each supplied test and compare its expected result without showing my offscreen guide."
ACTION	Click input 2.md in Explorer. Using ordinary window title bars and edges, move and resize Visual Studio Code to the left and Firefox to the right. Before each drag, name the window and edge you are moving. Keep both readable, avoid substantial overlap, and keep the menu bar and Dock visible. Do not enter full-screen Split View. If the screen cannot support readable side-by-side windows, use one ordinary active window at a time and narrate each switch.
SAY	"In Firefox on the right, I am reloading final output 1.html from the saved local file. The text area may retain old input; I will replace it with I01 before reviewing."
ACTION	Click Firefox, use its reload button, pause and point to the local-only note. If pressing Command-R instead, announce that shortcut.
CHECK	The table is the page's actual output, not a browser source tab. Both input files remain unchanged. Each new case below replaces the full text-area contents before clicking Review import.

8. Complete each acceptance group - about 20:30-32:00

ACTION	For each group, the selection is data only from the actual input 2.md. Do not select a Markdown fence, case title or Expected paragraph. Use visible Edit menus for Copy, Select All and Paste where practical; if using Command-C, Command-A or Command-V, name the shortcut when you use it. A clipboard transfer of supplied input data is not a transfer of completed solution code.
CHECK	Standard observation after every review: status, error list, preview row count, displayed IDs/locations/quantities and total where successful. A rejection must leave zero preview rows. Scroll in Firefox to the error list/table if necessary, naming which area you are bringing into view. Do not silently skip below-the-fold content.

8.01 I01 - Normalize a valid batch

SAY	"In Visual Studio Code's input 2.md on the left, I am scrolling to I01, normalize a valid batch. I am selecting only its input block so I can check the expected behavior in Firefox."
ACTION	Click the brief editor, scroll within it to the I01 heading and point to the expected result. Drag-select the literal input lines, excluding Markdown fences and explanation. Choose Edit > Copy. Keep the source unchanged.
SAY	"I am clicking the Inventory rows text area in Firefox on the right, selecting all of its previous input, and pasting I01's supplied text. I am clicking Review import below the text area to run a new local review."
ACTION	Focus the textarea, select all its contents, paste the copied data and point to the header and changed values. Click Review import, pause, and inspect the status, error list and table.
CHECK	Two preview rows: A101 / North store / 3 and B202 / Loading bay / 2. Status: Validated 2 rows (5 total units). Local review only; nothing imported. No errors.
IF WRONG	If I01 differs from its written expectation, say the case ID, app, expected result and actual result. Return to final output 1.html in Visual Studio Code; inspect the implicated parser, validator, renderer or review handler; correct only an actual problem; save; reload the output in Firefox; and repeat I01. If validation or reset behavior changed, rerun I01, I03 and I10 as relevant. Do not claim an unseen pass.

8.02 I02 - Zero and leading-zero quantities

SAY	"In Visual Studio Code's input 2.md on the left, I am scrolling to I02, zero and leading-zero quantities. I am selecting only its input block so I can check the expected behavior in Firefox."
ACTION	Click the brief editor, scroll within it to the I02 heading and point to the expected result. Drag-select the literal input lines, excluding Markdown fences and explanation. Choose Edit > Copy. Keep the source unchanged.
SAY	"I am clicking the Inventory rows text area in Firefox on the right, selecting all of its previous input, and pasting I02's supplied text. I am clicking Review import below the text area to run a new local review."
ACTION	Focus the textarea, select all its contents, paste the copied data and point to the header and changed values. Click Review import, pause, and inspect the status, error list and table.
CHECK	Two rows, quantities 0 and 3; total 3. Zero is valid. Leading zeros normalize to numeric display.
IF WRONG	If I02 differs from its written expectation, say the case ID, app, expected result and actual result. Return to final output 1.html in Visual Studio Code; inspect the implicated parser, validator, renderer or review handler; correct only an actual problem; save; reload the output in Firefox; and repeat I02. If validation or reset behavior changed, rerun I01, I03 and I10 as relevant. Do not claim an unseen pass.

8.03 I03 - Duplicate IDs after trimming and uppercasing

SAY	"In Visual Studio Code's input 2.md on the left, I am scrolling to I03, duplicate ids after trimming and uppercasing. I am selecting only its input block so I can check the expected behavior in Firefox."
ACTION	Click the brief editor, scroll within it to the I03 heading and point to the expected result. Drag-select the literal input lines, excluding Markdown fences and explanation. Choose Edit > Copy. Keep the source unchanged.
SAY	"I am clicking the Inventory rows text area in Firefox on the right, selecting all of its previous input, and pasting I03's supplied text. I am clicking Review import below the text area to run a new local review."
ACTION	Focus the textarea, select all its contents, paste the copied data and point to the header and changed values. Click Review import, pause, and inspect the status, error list and table.
CHECK	No preview rows. One error: Line 3: duplicate item ID A101.
IF WRONG	If I03 differs from its written expectation, say the case ID, app, expected result and actual result. Return to final output 1.html in Visual Studio Code; inspect the implicated parser, validator, renderer or review handler; correct only an actual problem; save; reload the output in Firefox; and repeat I03. If validation or reset behavior changed, rerun I01, I03 and I10 as relevant. Do not claim an unseen pass.

8.04 I04 - Missing location and missing ID

SAY	"In Visual Studio Code's input 2.md on the left, I am scrolling to I04, missing location and missing id. I am selecting only its input block so I can check the expected behavior in Firefox."
ACTION	Click the brief editor, scroll within it to the I04 heading and point to the expected result. Drag-select the literal input lines, excluding Markdown fences and explanation. Choose Edit > Copy. Keep the source unchanged.
SAY	"I am clicking the Inventory rows text area in Firefox on the right, selecting all of its previous input, and pasting I04's supplied text. I am clicking Review import below the text area to run a new local review."
ACTION	Focus the textarea, select all its contents, paste the copied data and point to the header and changed values. Click Review import, pause, and inspect the status, error list and table.
CHECK	No preview rows. Two errors: Line 2: location is required. / Line 3: item ID is required.
IF WRONG	If I04 differs from its written expectation, say the case ID, app, expected result and actual result. Return to final output 1.html in Visual Studio Code; inspect the implicated parser, validator, renderer or review handler; correct only an actual problem; save; reload the output in Firefox; and repeat I04. If validation or reset behavior changed, rerun I01, I03 and I10 as relevant. Do not claim an unseen pass.

8.05 I05 - Blank, fractional, negative and exponent quantities

SAY	"In Visual Studio Code's input 2.md on the left, I am scrolling to I05, blank, fractional, negative and exponent quantities. I am selecting only its input block so I can check the expected behavior in Firefox."
ACTION	Click the brief editor, scroll within it to the I05 heading and point to the expected result. Drag-select the literal input lines, excluding Markdown fences and explanation. Choose Edit > Copy. Keep the source unchanged.
SAY	"I am clicking the Inventory rows text area in Firefox on the right, selecting all of its previous input, and pasting I05's supplied text. I am clicking Review import below the text area to run a new local review."
ACTION	Focus the textarea, select all its contents, paste the copied data and point to the header and changed values. Click Review import, pause, and inspect the status, error list and table.
CHECK	No preview rows. Four quantity errors on physical lines 2, 3, 4 and 5. Each says quantity must be an integer from 0 to 9999.
SAY	"In Firefox's error list, I am checking all four source line numbers, two through five. Blank, one-point-five, negative one and one-e-two are all invalid quantity strings, even if Number could convert some of them."
ACTION	Point to each error. If the list is below view, announce a downward scroll in the Firefox page to the quantity errors, then scroll. Confirm that the preview table is empty.
IF WRONG	If I05 differs from its written expectation, say the case ID, app, expected result and actual result. Return to final output 1.html in Visual Studio Code; inspect the implicated parser, validator, renderer or review handler; correct only an actual problem; save; reload the output in Firefox; and repeat I05. If validation or reset behavior changed, rerun I01, I03 and I10 as relevant. Do not claim an unseen pass.

8.06 I06 - Wrong number of fields

SAY	"In Visual Studio Code's input 2.md on the left, I am scrolling to I06, wrong number of fields. I am selecting only its input block so I can check the expected behavior in Firefox."
ACTION	Click the brief editor, scroll within it to the I06 heading and point to the expected result. Drag-select the literal input lines, excluding Markdown fences and explanation. Choose Edit > Copy. Keep the source unchanged.
SAY	"I am clicking the Inventory rows text area in Firefox on the right, selecting all of its previous input, and pasting I06's supplied text. I am clicking Review import below the text area to run a new local review."
ACTION	Focus the textarea, select all its contents, paste the copied data and point to the header and changed values. Click Review import, pause, and inspect the status, error list and table.
CHECK	No preview rows. One error: Line 2: expected 3 fields.

IF WRONG	If I06 differs from its written expectation, say the case ID, app, expected result and actual result. Return to final output 1.html in Visual Studio Code; inspect the implicated parser, validator, renderer or review handler; correct only an actual problem; save; reload the output in Firefox; and repeat I06. If validation or reset behavior changed, rerun I01, I03 and I10 as relevant. Do not claim an unseen pass.

8.07 I07 - Wrong header

SAY	"In Visual Studio Code's input 2.md on the left, I am scrolling to I07, wrong header. I am selecting only its input block so I can check the expected behavior in Firefox."
ACTION	Click the brief editor, scroll within it to the I07 heading and point to the expected result. Drag-select the literal input lines, excluding Markdown fences and explanation. Choose Edit > Copy. Keep the source unchanged.
SAY	"I am clicking the Inventory rows text area in Firefox on the right, selecting all of its previous input, and pasting I07's supplied text. I am clicking Review import below the text area to run a new local review."
ACTION	Focus the textarea, select all its contents, paste the copied data and point to the header and changed values. Click Review import, pause, and inspect the status, error list and table.
CHECK	No preview rows. One error: Use header: item_id|location|quantity.
IF WRONG	If I07 differs from its written expectation, say the case ID, app, expected result and actual result. Return to final output 1.html in Visual Studio Code; inspect the implicated parser, validator, renderer or review handler; correct only an actual problem; save; reload the output in Firefox; and repeat I07. If validation or reset behavior changed, rerun I01, I03 and I10 as relevant. Do not claim an unseen pass.

8.08 I08 - Header without data rows

SAY	"In Visual Studio Code's input 2.md on the left, I am scrolling to I08, header without data rows. I am selecting only its input block so I can check the expected behavior in Firefox."
ACTION	Click the brief editor, scroll within it to the I08 heading and point to the expected result. Drag-select the literal input lines, excluding Markdown fences and explanation. Choose Edit > Copy. Keep the source unchanged.
SAY	"I am clicking the Inventory rows text area in Firefox on the right, selecting all of its previous input, and pasting I08's supplied text. I am clicking Review import below the text area to run a new local review."
ACTION	Focus the textarea, select all its contents, paste the copied data and point to the header and changed values. Click Review import, pause, and inspect the status, error list and table.
CHECK	No preview rows. One error: Enter at least one data row.

IF WRONG	If I08 differs from its written expectation, say the case ID, app, expected result and actual result. Return to final output 1.html in Visual Studio Code; inspect the implicated parser, validator, renderer or review handler; correct only an actual problem; save; reload the output in Firefox; and repeat I08. If validation or reset behavior changed, rerun I01, I03 and I10 as relevant. Do not claim an unseen pass.

8.09 I09 - Markup-like location is literal text

SAY	"In Visual Studio Code's input 2.md on the left, I am scrolling to I09, markup-like location is literal text. I am selecting only its input block so I can check the expected behavior in Firefox."
ACTION	Click the brief editor, scroll within it to the I09 heading and point to the expected result. Drag-select the literal input lines, excluding Markdown fences and explanation. Choose Edit > Copy. Keep the source unchanged.
SAY	"I am clicking the Inventory rows text area in Firefox on the right, selecting all of its previous input, and pasting I09's supplied text. I am clicking Review import below the text area to run a new local review."
ACTION	Focus the textarea, select all its contents, paste the copied data and point to the header and changed values. Click Review import, pause, and inspect the status, error list and table.
CHECK	One row, total 2; location displays the literal characters Store, rather than bold Store. No markup is executed.
SAY	"In the Firefox preview's Location column, I am checking that the angle brackets and b tags are visible characters. The word Store must not become formatted HTML."
ACTION	Point to the complete literal Store cell. Do not open an inspector or claim broader security testing. This check verifies the provided literal-text case.
IF WRONG	If I09 differs from its written expectation, say the case ID, app, expected result and actual result. Return to final output 1.html in Visual Studio Code; inspect the implicated parser, validator, renderer or review handler; correct only an actual problem; save; reload the output in Firefox; and repeat I09. If validation or reset behavior changed, rerun I01, I03 and I10 as relevant. Do not claim an unseen pass.

8.10 I10 - Valid-invalid-valid recovery

SAY	"In Visual Studio Code's input 2.md on the left, I am scrolling back to I01 for the first part of I10's recovery test. I will run a valid review, then I03's invalid duplicate review, then I01 again."
ACTION	Scroll to I01 in the brief. Select only its three-line data block, choose Edit > Copy, click Firefox's source text area, select all text there, paste and click Review import. Narrate each app focus, selection, copy, replacement and button click.
CHECK	First stage: exactly two rows, total five and no errors.

SAY	"I am returning to I03 in the left-hand brief, copying its duplicate-ID input, replacing Firefox's full text area and reviewing again. I am checking that no old successful preview remains."
ACTION	Scroll to I03, select its data only, copy, click the Firefox textarea, select all, paste and click Review import. Pause and point to the rejection and empty preview.
CHECK	Middle stage: exactly one duplicate error and no preview rows.
SAY	"I am returning to I01 in the brief and reviewing that valid batch once more in Firefox. This must clear the error and render exactly two rows rather than accumulate old rows."
ACTION	Repeat the explicit copy/replace/review steps using I01. Point to both row IDs and the cleared error area.
CHECK	Final stage: two rows, total five, no errors. If any stage differs, this sequence has not passed.
IF WRONG	If I10 differs from its written expectation, say the case ID, app, expected result and actual result. Return to final output 1.html in Visual Studio Code; inspect the implicated parser, validator, renderer or review handler; correct only an actual problem; save; reload the output in Firefox; and repeat I10. If validation or reset behavior changed, rerun I01, I03 and I10 as relevant. Do not claim an unseen pass.

8.11 I11 - Blank lines, trimmed header and line endings

SAY	"In Visual Studio Code's input 2.md on the left, I am scrolling to I11, blank lines, trimmed header and line endings. I am selecting only its input block so I can check the expected behavior in Firefox."
ACTION	Click the brief editor, scroll within it to the I11 heading and point to the expected result. Drag-select the literal input lines, excluding Markdown fences and explanation. Choose Edit > Copy. Keep the source unchanged.
SAY	"I am clicking the Inventory rows text area in Firefox on the right, selecting all of its previous input, and pasting I11's supplied text. I am clicking Review import below the text area to run a new local review."
ACTION	Focus the textarea, select all its contents, paste the copied data and point to the header and changed values. Click Review import, pause, and inspect the status, error list and table.
CHECK	Two normalized rows, total 5, with no error. Parser must also accept CRLF and an optional leading UTF-8 BOM. Verify those parser properties in source and in offscreen validation; do not claim a Firefox CRLF/BOM check unless the actual input preserves those characters.
CHECK	Preserve the leading and interior blank lines when selecting the actual I11 block. The on-camera copy/paste check verifies visible blank-line and trimming behavior. Only claim CRLF or BOM handling if you actually preserve and test those characters; otherwise verify the parser's split/BOM logic in source and keep that distinction in your reflection.

IF WRONG	If I11 differs from its written expectation, say the case ID, app, expected result and actual result. Return to final output 1.html in Visual Studio Code; inspect the implicated parser, validator, renderer or review handler; correct only an actual problem; save; reload the output in Firefox; and repeat I11. If validation or reset behavior changed, rerun I01, I03 and I10 as relevant. Do not claim an unseen pass.

8.12 I12 - Inclusive maximum and over-limit rejection

SAY	"In Visual Studio Code's input 2.md on the left, I am scrolling to I12, inclusive maximum and over-limit rejection. I am selecting only its input block so I can check the expected behavior in Firefox."
ACTION	Click the brief editor, scroll within it to the I12 heading and point to the expected result. Drag-select the literal input lines, excluding Markdown fences and explanation. Choose Edit > Copy. Keep the source unchanged.
SAY	"I am clicking the Inventory rows text area in Firefox on the right, selecting all of its previous input, and pasting I12's supplied text. I am clicking Review import below the text area to run a new local review."
ACTION	Focus the textarea, select all its contents, paste the copied data and point to the header and changed values. Click Review import, pause, and inspect the status, error list and table.
CHECK	First, 9999 is accepted as one row and total 9999. Then replace 9999 with 10000 and review again: no preview, one quantity error on Line 2.
SAY	"The Firefox review accepted 9999, which is the allowed maximum. In that same text area, I am selecting only 9999, replacing it with 10000 and clicking Review import again to check the first disallowed value."
ACTION	Select the quantity in the second source line, type 10000, click Review import, pause and point to the Line 2 quantity error and empty table.
CHECK	The maximum is inclusive: 9999 passes and 10000 rejects. The earlier maximum-value preview must disappear on rejection.
IF WRONG	If I12 differs from its written expectation, say the case ID, app, expected result and actual result. Return to final output 1.html in Visual Studio Code; inspect the implicated parser, validator, renderer or review handler; correct only an actual problem; save; reload the output in Firefox; and repeat I12. If validation or reset behavior changed, rerun I01, I03 and I10 as relevant. Do not claim an unseen pass.

9. Final saved-file and usable-state review - about 32:00-34:00

SAY	"I am leaving the completed page in a valid local-review state by returning to I01's input in the brief and reviewing it once more in Firefox. I will check the normalized IDs, two rows, five total units and local-only wording."

ACTION	Copy I01 data from the unchanged brief, replace the Firefox textarea and click Review import. Pause and point to the exact successful status and both preview rows. Narrate the copy, app focus, replacement and click.
SAY	"I am returning to final output 1.html in Visual Studio Code. I am checking the filename, saved state and the two separate original inputs before closing the workflow."
ACTION	Click Visual Studio Code, open the output tab and point to its name and absence of an unsaved indicator. Point to input 1.html and input 2.md in Explorer. Scroll to the completed validateBatch function, the start-of-review clearing line and safe renderer, naming each scroll destination.
CHECK	The final code has no unresolved syntax error, unintended request/storage code or unseen changes. The two inputs remain unchanged. Keep the exact saved file from this recording. If you change the output after the final Firefox review, save and retest before stopping.

10. Closing reflection - target 3-4 minutes, maximum 4:00

CHECK	These are conditional speaking cues. Use only outcomes verified in the current take. If the task was not complete, state the limitation. The cursor may remain still during this epilogue. Do not extend it beyond four minutes. A real correction can replace the optional paragraph; do not append unlimited text or invent an error.
SAY	"The deliverable is final output 1.html, a standalone inventory-import review page. It previews a synthetic batch locally and makes no change to inventory or stored data. I preserved the original starter and requirements. The final file now validates business values before showing a preview, and it clears earlier results when a new review starts."
SAY	"I began with the requirements rather than changing the display first. The starter already knew how to split the plain-text format, preserve source line numbers and render cells as text. The missing work was deciding which records could be accepted together. I kept that business validation separate from parsing and rendering, so the different responsibilities remain easy to inspect."
SAY	"The main identity decision was to uppercase each already-trimmed item ID before comparing it with the Set of IDs seen in this batch. A101 and a101 refer to the same item under this brief. Checking the raw spelling first would miss that duplicate. I also validate the raw quantity text, because numeric conversion alone would accept an empty string as zero and exponent syntax as a number. Zero itself is valid, so it needs an explicit rule rather than a truthiness test."
SAY	"In the baseline, the duplicate batch was accepted and a later header error left that earlier preview on screen. I used those observed defects to check the purpose of the repair. In the completed page, a failure leaves no preview rows, and a valid-invalid-valid sequence removes old rows and errors instead of accumulating them. That keeps the displayed result tied to the latest review."
SAY	"I compared the acceptance groups in Firefox with the written expectations, including missing values, invalid quantities, header and field-count errors, zero, leading zeros and the upper boundary. I also checked that the markup-like location stayed literal text. I retained textContent rather than interpreting record values as HTML. My local browser observations apply to these test inputs; they do not establish that every possible file or browser has been tested."

SAY	"For a production importer, I would add real backend validation, authorization and a controlled import transaction, plus broader automated coverage and a documented file-size limit. Those features are outside this local review task. The principle here is that parsing a record is different from accepting a batch. Normalize before comparing identity, validate before rendering success, and keep any reported errors traceable to their original source lines."
IF WRONG	If a real correction occurred, replace part of the reflection with its case ID, observed issue, exact correction and successful retest. If a requirement remains unverified, state that limitation instead of claiming completion. Do not read the CRLF/BOM result as browser-tested unless it actually was.

11. Stop and archive

SAY	"The saved output and local checks are complete. I am bringing TaskRecorder forward from the side Dock to stop this continuous recording."
ACTION	Click TaskRecorder in the side Dock, then Stop Recording. Wait through Stopping and Archiving until Idle. Do not quit, rename, edit or interrupt the recording archive.

12. After recording - OFF SCREEN

CHECK	Open the matching video.mp4 in QuickTime and inspect beginning, middle and end for the right attempt, usable picture and clear speech. Check the archive contains the corresponding screenshots, video.mp4, frames.jsonl, ax.jsonl, input.jsonl, processes.jsonl and recording.log. Screenshot rate must exceed 0.5 per recording second. Do not mix attempts.
ACTION	Build Deliverables.zip from unchanged actual input 1.html, unchanged actual input 2.md, the exact recorded final output 1.html and any actual intermediate artifacts. Open the ZIP and verify its contents are labeled and readable. Do not include this guide, planning docs, metadata or a rehearsal output.
ACTION	Create final Metadata.rtf after recording. Request is the exact approved Goal used in TaskRecorder. Topology lists every actual app visit in chronological order, including returns, using only LOOKUP, CREATE or MODIFY. Side-by-side test copying may produce many alternating Visual Studio Code and Firefox visits; do not replace that real sequence with the shorter proposal outline. Task Description is a numbered high-level account of the work actually done.
ACTION	Upload the untouched matching recorder ZIP, separate matching video.mp4, Deliverables.zip and separate Metadata.rtf in the same approved Studio task. For actual AI-prepared inputs, use the required Final Notes disclosure, AI-generated before recording, truthfully. Run both AutoQCs, resolve or dispute Major findings, and complete only manual checks you actually verified. Confirm submission status through Studio; a saved file alone is not submission.